

Aufgabe	1	2	3	4	5	Summe
Punkte	/ 25 P	/ 25 P	/ 14 P	/ 10 P	/ 26 P	/ 100 P

Für die Aufgaben sind alle schriftlichen Unterlagen und Taschenrechner zugelassen. Es wird angenommen, dass für die Aufgaben die gleichen Randbedingungen gelten wie für die im Praktikum benutzten STM32L476RG-Rechner.

Aufgabe 1 (25 Punkte)

Gegeben ist ein absichtlich nicht kommentiertes Assembler-Programm für den Micro-Controller STM32L476RG.

```
.syntax unified
.global musteraufg1
.text
@ Hauptprogramm ab Adresse 0x08000000 im Flash-Speicher
musteraufg1:    ldr sp, =0x20016FFF
                adr r0, table
                ldr r1, =#0x20000000
                ldr r2, =0x0
                ldr r3, =0x8
                @----> a)
                bl up1
ende:          nop
                b ende
@ Unterprogramm
up1:           ldr r4, [r0], #4
                ands r4, #0x80000000
                beq up2
                ldr r4, [r0, #-4]
                str r4, [r1], #4
                add r2, #1
up2:           subs r3, #1
                bne up1
                str r2, [r1], #1
                @-----> b)
                bx lr
@ Daten ab Adresse 0x08000204 im Flash-Speicher
table:         .word 0x80F0C90F, 0xEA3F5612
                .word 0x10E0309C, 0x327FAA37
                .word 0x62FF7400, 0xC3F8D165
                .word 0x1FEA7500, 0x40CF3443
```

Geben Sie die Inhalte der Register an:

a) direkt vor Ausführung des Befehls **b1 up10** im Hauptprogramm,

r0	r1	r2	sp

b) direkt vor Ausführung des Befehls **bx 1r** im Unterprogramm.

r0	r1	r2	sp

c) Welche Zahlen (in hexadezimaler Darstellung) stehen im Speicher ab der Adresse **0x20000000**, nachdem das Programm abgearbeitet ist? Tragen Sie die Zahlen als 32 bit Zahlen in die Tabelle ein.

0x20000000	Daten (32 Bit)

Aufgabe 2 (25 Punkte)

Ein Programm soll eine Tabelle (**tab1**) in eine andere Tabelle (**tab2**) umkopieren. Auf die Tabellen zugreifen kann das Programm mit Hilfe der Register r0 und r1. In der über Zeiger r0 erreichbaren Tabelle tab1 stehen vorzeichenbehaftete 32-Bit-Zahlen (positive Zahlen, negative Zahlen und Nullen). Insgesamt enthält die Tabelle 64 solcher Zahlen. Zunächst sollen die negativen Zahlen in die über Register r1 erreichbare Tabelle tab2 kopiert werden. Danach sind die positiven Zahlen aus tab1 an tab2 anzuhängen. In der neuen Tabelle tab2 stehen nach dem Umkopieren die negativen Zahlen von tab1 und dahinter die positiven Zahlen von tab1. Innerhalb der Blöcke (negative Zahlen / positive Zahlen) soll nicht sortiert werden. Falls die Zahl 0 in der Tabelle gefunden wird, darf diese nicht mit kopiert werden. In dem Register r3 soll nach dem Umkopieren die Anzahl der Tabellenelemente von tab2 stehen. Schreiben Sie ein Programm in stm32-Assembler, das die Zahlen kopiert. Die ersten beiden Codezeilen sind vorgeben, vervollständigen Sie Ihr Programm gemäß der Aufgabenstellung.

Labelfeld	Operationsfeld	Operandenfeld	Kommentarfeld
	adr	r0, tab1	Anfangsadresse Tabelle 1
	adr	r1, tab2	Anfangsadresse Tabelle 2

...

tab1:	.word	...	

Aufgabe 3 (14 Punkte)

Der Pointer `char *pe2_ptr`; soll auf die absolute Adresse (hex)21000000 zeigen und der Pointer `char *pe3_ptr`; auf (hex)24000000.

Mit Hilfe der Pointer sollen 100 Bytes, die ab Adresse (hex)21000000 im Speicher stehen, auf die Adressen (hex)24000000 und folgende kopiert werden. Falls die Zahl 0x00 kopiert wurde, ist der Kopiervorgang vorzeitig abubrechen.

Schreiben sie zunächst die Initialisierung der Pointer und anschließend den Kopiervorgang.

Initialisierung der Pointer.

Kopiervorgang.

Aufgabe 4 (10 Punkte)

Ein Peripheriemodul für die digitale Ausgabe (nicht identisch mit einem Modul aus dem Praktikum!) besitzt mehrere Register (jeweils 32 Bit Breite, Datentyp long). Drei dieser Register benötigen Sie, um mit dem Baustein arbeiten zu können.

output type auf Adresse 0x03800040,
output speed register auf Adresse 0x03800044,
output data register auf Adresse 0x0380004C,

Definieren Sie einen neuen Datentyp (Strukturmaske), mit dem Sie auf den Baustein zugreifen können und initialisieren Sie den Peripherie-Baustein. Die Initialisierungs-Werte sind:

0x000000FF für das output type register

0x55005500 für das output speed register

Definition Datentyp

Initialisierung Peripherie

Aufgabe 5 (26 Punkte)

Aus dem I/O-Register eines Peripheriemoduls (nicht identisch mit einem Modul aus dem Praktikum!) wird ein 8-Bit-Messwert gelesen (auf Adresse 0x04800000).

Vereinfachungen: das Peripheriemodul ist bereits initialisiert, die Digitaleingabe ist entprellt.

Nach jeder Änderung des Messwertes sollen Teile des 8-Bit-Wertes abgespeichert werden.

Der Messwert ist in folgende Werte aufzuteilen:

- **Wert 1:** Bit 0 bis Bit 3 sollen rechtsbündig in einem Byte stehen.
- **Wert 2:** Bit 4 bis Bit 6 sollen rechtsbündig in einem Byte stehen.

100 Messungen sollen in einem Feld aus Strukturen (100 Feldelemente) abgelegt werden. Die Strukturkomponenten sind: `short int nummer; char wert_1; char wert_2;`

Schreiben Sie ein C-Programm, das den Messwert einliest, in zwei Werte aufteilt, abspeichert und jeweils mit einer laufenden Nummer versieht. (Schreiben Sie die Definitionen von Variablen und Datentypen und den Programmcode in den Kasten.

